PHẦN 02 / 03

02

Computer vision & Hỗ trợ gắp

Nguyễn Anh Dương – Phạm Trương Vũ

PHẦN NÀY GỒM

01 Sơ đồ luồng dữ liệu

02 Vòng lặp mỗi khung hình · main.py

03 Tìm camera theo tên · cameras.py

04 Hệ tọa độ từ tờ A4 · vision.py

05 Nhận diện vật · vision.py

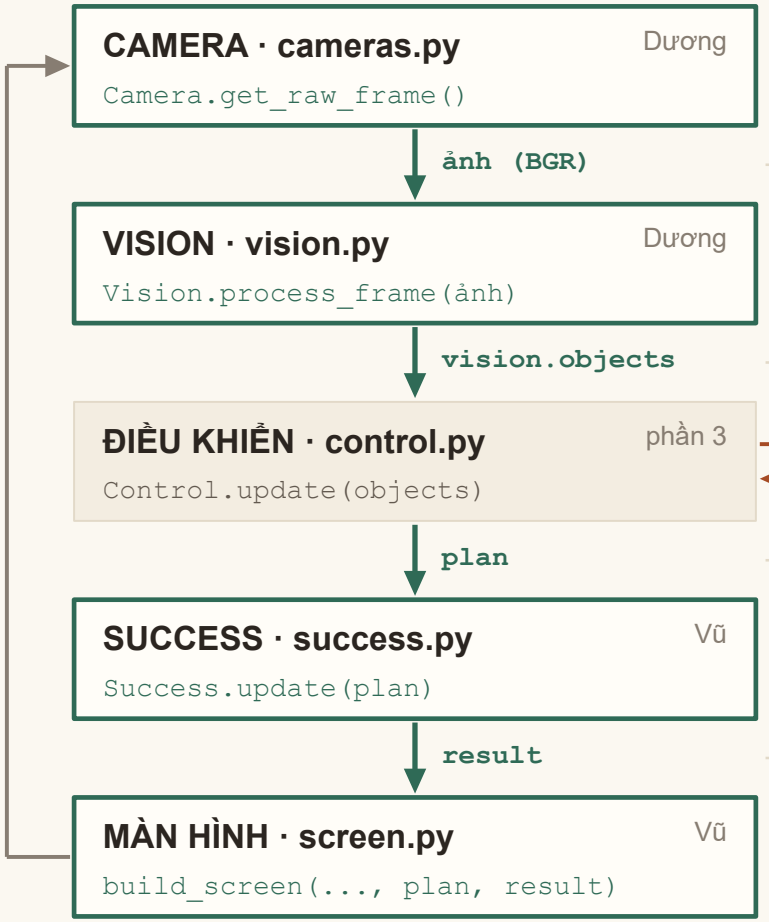
06 Điều kiện hoàn thành · success.py

07 Màn hình · screen.py

08 Thử nghiệm, kết quả, hạn chế

Nguyễn Anh Dương · Computer vision

Sơ đồ luồng dữ liệu



Arduino · USB 9600

phím ↓ · JSON góc ↑

ảnh

mảng numpy cao × rộng × 3 (BGR), mỗi camera một ảnh; cam TRÊN để tìm vật, cam NGANG để người nhìn

vision.objects

danh sách vật [{"name", "x_cm", "y_cm", "px"}]: tọa độ trên tờ A4 (cm), đã bù thị sai

Plan

now: 4 góc servo hiện tại · goal: 3 góc cần tới · moves: phím, số lần, hướng, độ · found, reachable

Result

state, done, joints_ok, next_move, remaining (số lần bấm còn lại), count

Lặp lại mỗi khung hình: **main.py · App.render()**

Nguyễn Anh Dương · Computer vision

Vòng lặp mỗi khung hình · main.py

1 Chỉ xử lý khi có khung mới 2 vision 3 control → Plan 4 success → Result 5 Vẽ màn hình

main.py · dòng 60–71, 82–89

```
60  def _frame(self, cam, mode):
61      """Khung đã vẽ lớp phủ của một camera. Chỉ xử lý nặng (ArUco + tìm vật) khi camera có khung mới."""
1   62      cached = self.cache.get(mode)
63      if cam.stalled() or cached is None or cached["id"] != cam.frame_id:
64          frame_id, raw = cam.frame_id, cam.get_raw_frame()
65          if raw is None:  # Chưa có hình hoặc mất tín hiệu: hiện "Dang cho tin hieu" thay vì ảnh đứng hình, quên vật cũ
66              self.cache.pop(mode, None)
67              if mode == "topdown":
68                  self.vision.objects = []
69              return cam.waiting_image()
2   70          cached = self.cache[mode] = {"id": frame_id, "annotated": self.vision.process_frame(raw, mode=mode)}
71      return cached["annotated"]
    ...
82  def render(self):
83      """Dựng một khung màn hình: (ảnh màn hình, Plan của control, Result của success)."""
84      left, right = self._frame(self.cam_side, "side"), self._frame(self.cam_top, "topdown")
3   85      plan = self.control.update(self.vision.objects)  # Khối điều khiển: vật ở đâu -> góc cần tới -> bấm phím gì
4   86      result = self.success.update(plan)                # Khối success: đã tới chưa
87      titles = (f"CAM NGANG: {self.cam_side.label}", f"CAM TREN: {self.cam_top.label} (nhin tu tren)")
5   88      screen = build_screen(self._with_target(left, "side", plan), self._with_target(right, "topdown", plan), plan, result, titles=titles)
89      return screen, plan, result
```

Đọc camera trên luồng riêng · cameras.py

1 Mở camera theo chỉ số

2 Lưu khung mới nhất

3 Mất tín hiệu 4 giây: mở lại

cameras.py · dòng 35–44, 54–66

```
35  def _open(self):
36      if self.cap is not None:
37          self.cap.release()
38      self.cap = self.image = None
39      self.opened_time = time.time()
40      if self.arg.lower().endswith((".png", ".jpg", ".jpeg")):  # Ảnh tĩnh: lặp lại mãi một khung
41          self.image = cv2.imread(self.arg)
1   42      elif self.arg.isdigit():
43          index = int(self.arg)
44          self.cap = cv2.VideoCapture(index, cv2.CAP_DSHOW) if sys.platform == "win32" else cv2.VideoCapture(index)
    ...
54  def _loop(self):
55      self._open()
56      while self.running:
3   57          if self.stalled() and time.time() - self.opened_time > RETRY_SECONDS:  # Mất tín hiệu: thử mở lại sau mỗi
                    RETRY_SECONDS
58              self._open()
59          frame = self._read()
60          if frame is None:
61              time.sleep(0.1)
62              continue
63          with self.lock:
2   64              self.frame, self.frame_id, self.last_time = frame, self.frame_id + 1, time.time()
65      if self.cap is not None:
66          self.cap.release()
```

Nguyễn Anh Dương · Computer vision

Hệ tọa độ từ tờ A4 · vision.py

1 Góc mã theo cm 2 Dò mã ArUco 3 Nhớ mã bị che 4 Tính homography

vision.py · dòng 26–31, 45–62

```
1  26  def marker_corners_cm(i):
   27      """4 góc (cm) của mã ID i (0-3) theo thứ tự OpenCV trả về: trên-trái, trên-phải, dưới-phải, dưới-trái."""
   28      s, m = MARKER_SIZE_CM, MARKER_MARGIN_CM
   29      x = m if i in (0, 3) else SHEET_W_CM - m - s
   30      y = m if i in (0, 1) else SHEET_H_CM - m - s
   31      return np.float32([[x, y], [x + s, y], [x + s, y + s], [x, y + s]])
   ...
   45  class Sheet:
   46      def __init__(self):
   47          self.markers = {}  # ID mã -> 4 góc pixel lần cuối thấy (mã đang bị che thì dùng vị trí đã nhớ)
   48          self.n_seen = 0    # Số mã thấy trong khung vừa xử lý
   49          self.H_px2cm = self.H_cm2px = None
   50
   51      def update(self, frame):
2  52          corners, ids, _ = DETECTOR.detectMarkers(frame)
   53          ids = [] if ids is None else ids.ravel()
   54          seen = {int(i): c.reshape(4, 2) for c, i in zip(corners, ids) if i in (0, 1, 2, 3)}
3  55          self.markers.update(seen)
   56          self.n_seen = len(seen)
   57          if len(self.markers) >= 2:  # 2 mã = 8 góc, đủ tính homography
   58              px = np.concatenate(list(self.markers.values()))
   59              cm = np.concatenate([marker_corners_cm(i) for i in self.markers])
4  60              self.H_px2cm = cv2.findHomography(px, cm)[0]
   61              self.H_cm2px = np.linalg.inv(self.H_px2cm)
   62          return corners
```

Nhận diện vật

BƯỚC 1

Ảnh phẳng tờ A4

Homography, 8 pixel / cm

BƯỚC 2

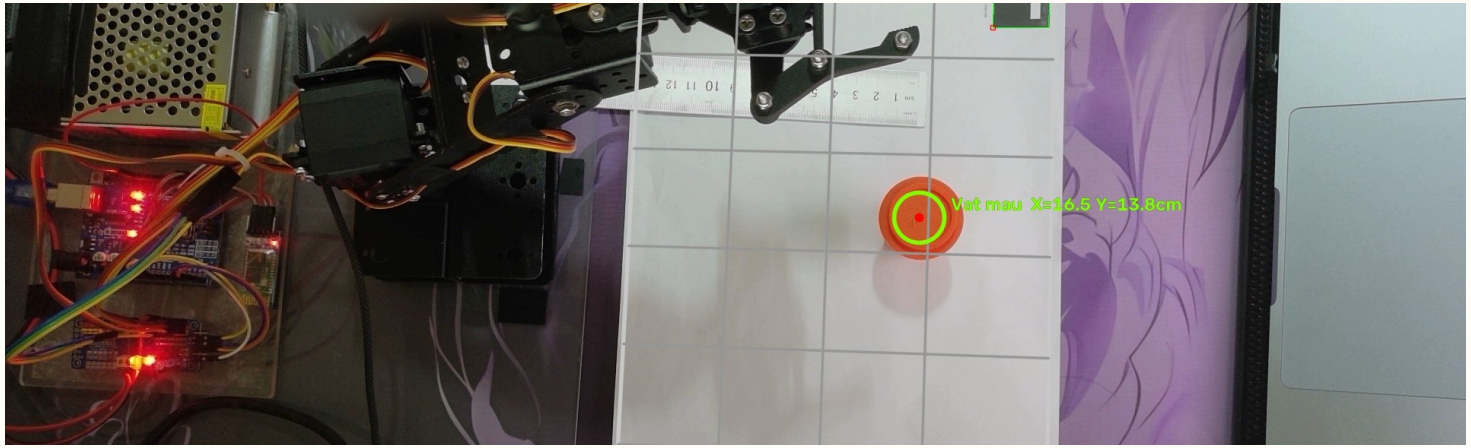
Tách vùng có màu

Độ bão hòa ≥ 90, đủ sáng

BƯỚC 3

Lọc vật

Bỏ khối < 1,5 cm² hoặc > 50 cm², sát mép giấy, dính thân robot



Cam TRÊN, ảnh đã lưu: vòng xanh quanh cốc kèm X, Y (cm) (giao diện bản trước)

CHỌN MỤC TIÊU

- Ưu tiên vật gần đế nhất
- **R** chụp vị trí làm đích
- Góc cần tới tính một lần
- Vật bị dời: bấm **R** lại

Nhận diện vật · vision.py

1 Ảnh phẳng tờ A4 2 Tách vùng có màu 3 Lọc vật 4 Bù thị sai

vision.py · dòng 68–95

```python
def find_objects(frame, sheet):
    """Danh sách vật [{"name", "x_cm", "y_cm", "px"}]; px là tâm vật trên ảnh, để vẽ."""
    W, H = round(SHEET_W_CM * PPC), round(SHEET_H_CM * PPC)
    flat = cv2.warpPerspective(frame, np.diag([PPC, PPC, 1.0]) @ sheet.H_px2cm, (W, H))  # Ảnh phẳng tờ giấy: 1 cm = PPC pixel
    hsv = cv2.cvtColor(flat, cv2.COLOR_BGR2HSV)
    colored = cv2.inRange(hsv, (0, 90, 60), (180, 255, 255))                    # Có màu: độ bão hòa >= 90, đủ sáng
    colored = cv2.morphologyEx(colored, cv2.MORPH_OPEN, np.ones((5, 5), np.uint8))  # Bỏ nhiễu và dây điện mảnh
    robot = cv2.dilate(cv2.inRange(hsv, (0, 0, 0), (180, 255, 69)), np.ones((9, 9), np.uint8))  # Phần đen của cánh tay, nới thêm
        ~1 cm

    h, w = frame.shape[:2]
    nadir = px_to_cm(sheet.H_px2cm, w / 2, h / 2)  # Điểm cam TREN nhìn thẳng xuống = tâm ảnh
    k = (CAM_HEIGHT_CM - OBJECT_HEIGHT_CM) / CAM_HEIGHT_CM if CAM_HEIGHT_CM > 0 else 1.0
    objects = []
    for c in cv2.findContours(colored, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)[0]:
        x, y, bw, bh = cv2.boundingRect(c)
        if not 1.5 <= cv2.contourArea(c) / PPC**2 <= 50:
            continue  # Nhỏ hơn 1,5 cm² là nhiễu; to hơn 50 cm² là bàn tay, không lọt miệng kẹp
        if x < PPC or y < PPC or x + bw > W - PPC or y + bh > H - PPC:
            continue  # Sát mép giấy (dưới 1 cm): tay hoặc đồ vật thò từ ngoài vào
        blob = cv2.drawContours(np.zeros_like(colored), [c], -1, 255, -1)
        if cv2.mean(robot, mask=blob)[0] > 0.3 * 255:
            continue  # Hơn 30% dính vào phần đen của cánh tay: dây điện, nhãn servo
        m = cv2.moments(c)
        top = (m["m10"] / m["m00"] / PPC, m["m01"] / m["m00"] / PPC)  # Tâm khối màu (cm) = nắp vật nhìn từ trên
        # Bù thị sai: vị trí thật = nadir + (vị trí đo - nadir) * (H - h) / H
        x_cm, y_cm = (nadir[j] + (top[j] - nadir[j]) * k for j in (0, 1))
        objects.append({"name": "Vat mau", "x_cm": x_cm, "y_cm": y_cm, "px": cm_to_px(sheet.H_cm2px, *top)})
    return objects
```

Điều kiện hoàn thành · success.py

1. Lệch ≤ 2°, giữ 1 giây
2. Servo nào đã tới
3. Trạng thái
4. Đếm lần thành công

success.py · dòng 11–12, 25–37, 48–58

```
11  TOLERANCE_STEPS = 1  # Lệch không quá 1 bước bấm (2°) là "đã tới": mô hình hình học chỉ chính xác cỡ đó
12  HOLD_S = 1.0         # Giữ đúng liên tục 1 giây mới đếm 1 lần thành công (khỏi đếm khi chỉ lướt qua)
    ...
25  def check(plan):
26      """Đánh giá một Plan (không nhớ gì giữa hai lần gọi)."""
27      joints = [m.degrees <= TOLERANCE_STEPS * m.step for m in plan.moves]
28      pending = [m for m, ok in zip(plan.moves, joints) if not ok]
29      if plan.relaxed:
30          state = "relaxed"  # Tay đã ngắt xung: góc báo về chỉ là góc lệnh cuối, không tin được
31      elif not plan.found:
32          state = "no_target"
33      elif not plan.reachable:
34          state = "unreachable"
35      else:
36          state = "moving" if pending else "done"
37      return Result(state, state == "done", joints, pending[0] if pending else None, sum(m.steps for m in pending))
    ...
48      def update(self, plan, now=None):
49          now = time.time() if now is None else now
50          result = check(plan)
51          if result.done:
52              self.since = now if self.since is None else self.since
53              if not self.counted and now - self.since >= HOLD_S:
54                  self.count, self.counted = self.count + 1, True
55          else:
56              self.since, self.counted = None, False
57          result.count = self.count
58          return result
```

Màn hình · screen.py

1. Hai camera
2. Ô 1: lệnh bấm phím
3. Ba dòng servo
4. Ô 2: XONG / CHUA XONG

screen.py · dòng 46–71

```
46  def build_screen(view_left, view_right, plan, result, titles=("CAM NGANG", "CAM TREN")):
47      img = np.full((SCREEN_H, SCREEN_W, 3), 30, np.uint8)
48      img[:CAM_H] = np.hstack([cam_cell(view_left, titles[0]), cam_cell(view_right, titles[1])])
49      y = CAM_H + 40
50
51      # Ô 1: ĐIỀU KHIỂN
52      put(img, "1. DIEU KHIEN (trai / phai nhin theo cam NGANG)", 20, y, 0.8, ORANGE)
53      if result.state == "moving":
54          m = result.next_move
55          put(img, f"BAM {m.key.upper()} x{m.steps}: {m.words} {m.degrees} do", 20, y + 50, 1.2, ORANGE, 3)
56      else:
57          put(img, MESSAGES[result.state], 20, y + 50)
58      for i, (m, ok) in enumerate(zip(plan.moves, result.joints_ok)):  # Đế, vai, khuỷu
59          todo = "DA DUNG" if ok else f"bam {m.key.upper()} x{m.steps}"
60          put(img, f"{m.label}: {m.now} -> {m.goal} do   {todo}", 20, y + 100 + 36 * i, 0.7, GREEN if ok else WHITE)
61
62      # Ô 2: SUCCESS
63      x = 840
64      put(img, "2. SUCCESS", x, y, 0.8, GREEN if result.done else ORANGE)
65      if result.done:
66          put(img, "XONG!", x, y + 70, 2.0, GREEN, 5)
67      else:
68          put(img, "CHUA XONG", x, y + 60, 1.2, ORANGE, 3)
69          if result.state == "moving":
70              put(img, f"con {result.remaining} lan bam", x, y + 100)
71      put(img, f"Thanh cong: {result.count} lan", x, y + 150)
```